Institución Universitaria Pascual Bravo
Ingeniería de Software

Tarea 5

Consultas DML y vistas (VIEWS)

Red social estudiantil pascualina

Curso: Bases de Datos I

Estudiante: Julián Quintero Copete

Grupo: 6

SGBD: PostgreSQL 17 (pgAdmin 4)

9 de Noviembre de 2026

Introducción

Este informe documenta la tercera unidad del reto de la red social estudiantil pascualina: la interacción con los datos mediante el lenguaje de manipulación de datos (DML) y la creación de vistas. Se trabaja sobre el modelo físico implementado en PostgreSQL 17, en la base de datos **red_social_pascualina**, esquema **red_social**.

El informe presenta el diagrama del modelo físico, la forma de trabajo, **siete (7) consultas DML** que responden preguntas de negocio, **cuatro (4) vistas** reutilizables, las dificultades encontradas y las conclusiones. Los scripts completos y comentados se entregan en el repositorio Git junto a este informe (*01_datos_prueba.sql* y *02_consultas_y_vistas.sql*).

Para que las consultas devolvieran resultados, se cargó un conjunto de datos de prueba ficticios: 8 estudiantes, 8 intereses, 13 seguimientos, 9 publicaciones, 10 comentarios, 4 grupos, 5 eventos y sus tablas puente. La Figura 1 muestra el conteo de filas por tabla.

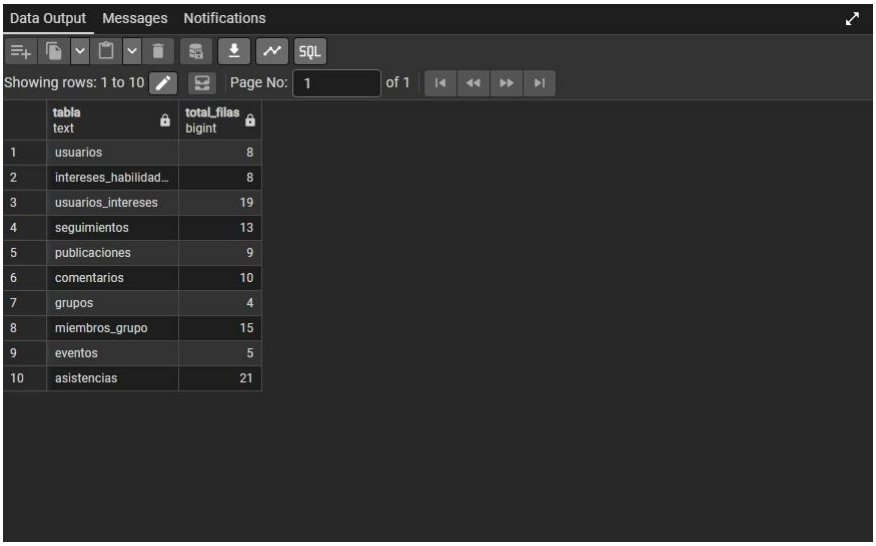


	tabla text	total_filas bigint
1	usuarios	8
2	intereses_habilidad...	8
3	usuarios_intereses	19
4	seguimientos	13
5	publicaciones	9
6	comentarios	10
7	grupos	4
8	miembros_grupo	15
9	eventos	5
10	asistencias	21

a. Diagrama del modelo físico de la base de datos

El modelo físico cuenta con 10 tablas. Las tablas *usuarios_intereses*, *seguimientos*, *miembros_grupo* y *asistencias* son tablas puente que resuelven relaciones muchos a muchos y usan clave primaria compuesta. Las columnas de ID de las demás tablas son de identidad (*GENERATED ALWAYS*). La Figura 2 corresponde al diagrama generado por pgAdmin (ERD).



Resumen de tablas, campos, tipos de datos y claves:

Tabla	Campos y tipos de datos	Claves
usuarios	id_usuario integer; nombre varchar(100); correo varchar(150), único; carrera varchar(80)	PK: id_usuario
intereses_habilidades	id_interes integer; nombre varchar(60), único	PK: id_interes
usuarios_intereses	id_usuario integer; id_interes integer	PK compuesta (id_usuario, id_interes); FK a usuarios e intereses_habilidades
seguimientos	id_seguidor integer; id_seguido integer	PK compuesta (id_seguidor, id_seguido); ambas FK a usuarios
publicaciones	id_publicacion integer; contenido text; fecha_publicacion timestamp; id_usuario_autor integer	PK: id_publicacion; FK: id_usuario_autor
comentarios	id_comentario integer; contenido varchar(500); id_usuario_autor integer; id_publicacion integer	PK: id_comentario; FK: id_usuario_autor, id_publicacion

grupos	id_grupo integer; nombre varchar(100); descripcion varchar(300); id_usuario_creador integer	PK: id_grupo; FK: id_usuario_creador
miembros_grupo	id_usuario integer; id_grupo integer	PK compuesta (id_usuario, id_grupo); FK a usuarios y grupos
Tabla	**Campos y tipos de datos**	**Claves**
eventos	id_evento integer; nombre varchar(100); fecha_evento timestamp; lugar varchar(150); id_usuario_organizador integer; id_grupo integer (opcional); detalles_json jsonb	PK: id_evento; FK: id_usuario_organizador, id_grupo
asistencias	id_usuario integer; id_evento integer	PK compuesta (id_usuario, id_evento); FK a usuarios y eventos

b. Explicación del trabajo en equipo

El reto está planteado para trabajarse en equipo (grupo 6). Sin embargo, esta tarea se desarrolló de **forma individual**, igual que las entregas anteriores del reto. A continuación se describe con transparencia cómo se organizó el trabajo.

Distribución del trabajo

- Preparación de los datos de prueba para las 10 tablas del modelo.
- Definición de las preguntas de negocio y escritura de las 7 consultas DML.
- Diseño de las 4 vistas y de sus consultas de prueba y reutilización.
- Ejecución en pgAdmin 4, captura de resultados y redacción del informe.

Proceso de discusión y depuración de ideas

Al no haber discusión con un equipo, la depuración se hizo contra el propio SGBD: cada script se ejecutó, se revisaron los mensajes de error y se corrigió hasta obtener los resultados esperados. Los errores encontrados se explican en la sección e.

Participación en el foro

No fue posible participar en el foro *Trabajando en la red social estudiantil* dentro del plazo: el foro cerró antes de que se pudiera aportar, debido a compromisos laborales del estudiante. Se reconoce como una limitación de esta entrega y se plantea una mejora en la sección f.

c. Consultas DML significativas

Convención de alias usada en todas las consultas: las columnas calculadas llevan nombres en minúscula con guion bajo (*snake_case*); los conteos usan el prefijo **total_** y los promedios el prefijo **promedio_**; los alias de tabla son abreviaturas cortas (u, p, c, g, e, ih, ui, mg, a, s).

Consulta 1: Estudiantes por interés o habilidad

Pregunta de negocio

¿Cuántos estudiantes están interesados en cada interés o habilidad?

Sentencia SQL

```
SELECT
    ih.nombre              AS interes,
COUNT(ui.id_usuario)   AS total_estudiantes
FROM intereses_habilidades ih
LEFT JOIN usuarios_intereses ui ON ui.id_interes = ih.id_interes
GROUP BY ih.id_interes, ih.nombre
ORDER BY total_estudiantes DESC, interes;
```

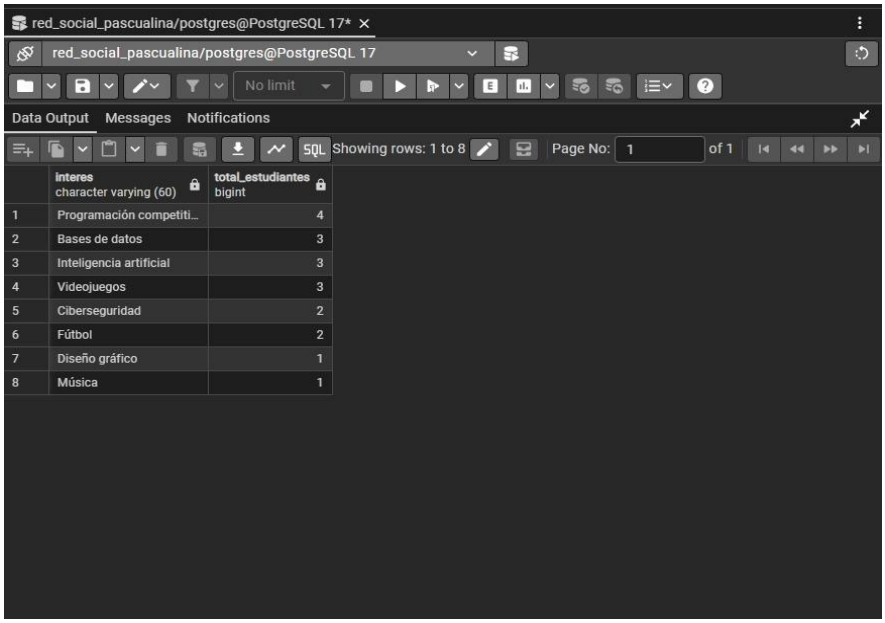
Explicación detallada

Parte de *intereses_habilidades* y la une con la tabla puente *usuarios_intereses* mediante **LEFT JOIN**, de modo que también aparezcan intereses sin estudiantes. **COUNT** cuenta los estudiantes de cada interés, **GROUP BY** agrupa por interés y **ORDER BY** ordena de mayor a menor. Resultado: *Programación competitiva* es el interés más frecuente, con 4 estudiantes.

Justificación de ALIAS

interes nombra de forma clara el campo *nombre*, que de otro modo sería ambiguo frente al nombre de otras tablas. **total_estudiantes** indica con el prefijo *total_* que es un conteo, y mejora la legibilidad frente al nombre por defecto *count*.

Captura del resultado



	interes character varying (60)	total_estudiantes bigint
1	Programación competiti...	4
2	Bases de datos	3
3	Inteligencia artificial	3
4	Videojuegos	3
5	Ciberseguridad	2
6	Fútbol	2
7	Diseño gráfico	1
8	Música	1

Consulta 2: Estudiantes con más seguidores

Pregunta de negocio

¿Qué estudiantes tienen más seguidores y podrían ser mentores?

Sentencia SQL

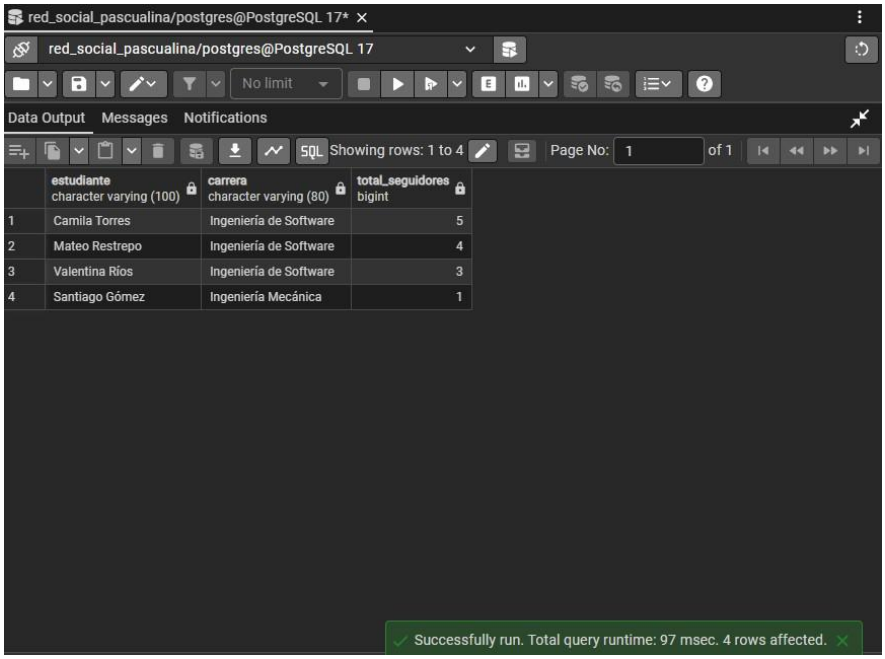
```
SELECT
    u.nombre                AS estudiante,
    u.carrera               AS carrera,
COUNT(s.id_seguidor)     AS total_seguidores
FROM usuarios u
LEFT JOIN seguimientos s ON s.id_seguido = u.id_usuario
GROUP BY u.id_usuario, u.nombre, u.carrera
HAVING COUNT(s.id_seguidor) >= 1
ORDER BY total_seguidores DESC, estudiante;
```

Explicación detallada

Une *usuarios* con *seguimientos* por el campo *id_seguido* (a quién siguen) y cuenta los seguidores de cada estudiante. **HAVING** filtra los grupos con al menos un seguidor, ya que **WHERE** no puede filtrar resultados de funciones de agregación. Resultado: Camila Torres (5), Mateo Restrepo (4), Valentina Ríos (3) y Santiago Gómez (1).

Justificación de ALIAS

estudiante y **carrera** reemplazan los nombres genéricos de las columnas; **total_seguidores** deja claro que es un conteo de seguidores y no de seguidos.



Consulta 3: Actividad por estudiante

Pregunta de negocio

¿Qué tan activo es cada estudiante, sumando sus publicaciones y comentarios?

Sentencia SQL

```
WITH actividad AS (
SELECT
        u.nombre AS estudiante,
(SELECT COUNT(*) FROM publicaciones p
          WHERE p.id_usuario_autor = u.id_usuario) AS total_publicaciones,
         (SELECT COUNT(*) FROM comentarios c
          WHERE c.id_usuario_autor = u.id_usuario)  AS total_comentarios
    FROM usuarios u
) SELECT
estudiante,
total_publicaciones,
total_comentarios,
    total_publicaciones + total_comentarios AS total_actividad
FROM actividad
ORDER BY total_actividad DESC, estudiante;
```

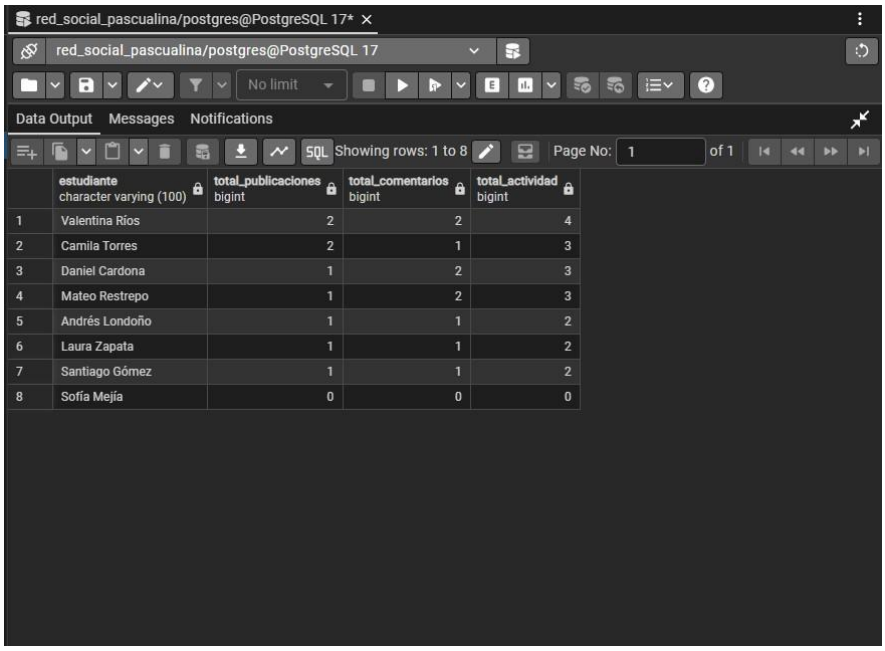
Explicación detallada

Usa una expresión de tabla común (**WITH**) llamada *actividad* y dos **subconsultas correlacionadas** que cuentan por separado las publicaciones y los comentarios de cada estudiante. Se hace por separado para evitar el doble conteo que ocurriría al unir ambas tablas a la vez. La consulta externa suma las dos columnas calculadas. Resultado: Valentina Ríos es la más activa (4) y Sofía Mejía no tiene actividad (0).

Justificación de ALIAS

total_publicaciones, **total_comentarios** y **total_actividad** permiten reutilizar los resultados en la suma final y se leen sin necesidad de ver el código interno.

Captura del resultado



	estudiante character varying (100)	total_publicaciones bigint	total_comentarios bigint	total_actividad bigint
1	Valentina Ríos	2	2	4
2	Camila Torres	2	1	3
3	Daniel Cardona	1	2	3
4	Mateo Restrepo	1	2	3
5	Andrés Londoño	1	1	2
6	Laura Zapata	1	1	2
7	Santiago Gómez	1	1	2
8	Sofía Mejía	0	0	0

Consulta 4: Publicaciones con más comentarios

Pregunta de negocio

¿Cuáles son las 5 publicaciones que generan más conversación?

Sentencia SQL

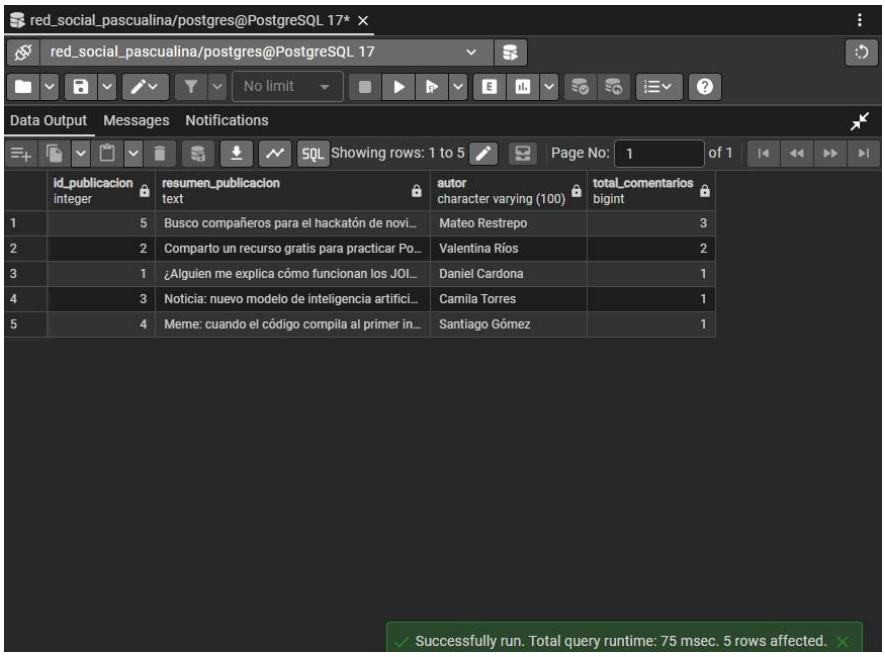
```
SELECT
    p.id_publicacion          AS id_publicacion,
LEFT(p.contenido, 50)     AS resumen_publicacion,
    u.nombre                  AS autor,
COUNT(c.id_comentario)    AS total_comentarios
FROM publicaciones p
JOIN usuarios u         ON u.id_usuario = p.id_usuario_autor
LEFT JOIN comentarios c ON c.id_publicacion = p.id_publicacion
GROUP BY p.id_publicacion, p.contenido, u.nombre
ORDER BY total_comentarios DESC, p.id_publicacion
LIMIT 5;
```

Explicación detallada

Combina tres tablas: *publicaciones*, *usuarios* (JOIN para obtener el autor) y *comentarios* (LEFT JOIN para no perder publicaciones sin comentarios). La función **LEFT** recorta el contenido a 50 caracteres, **COUNT** cuenta los comentarios y **LIMIT 5** devuelve solo las cinco primeras. Resultado: la publicación 5 (búsqueda de compañeros para el hackatón) lidera con 3 comentarios.

Justificación de ALIAS

resumen_publicacion aclara que el texto está recortado, **autor** da un nombre de negocio al nombre del usuario y **total_comentarios** sigue la convención de conteos.



Consulta 5: Grupos con más miembros

Pregunta de negocio

¿Qué grupos tienen más miembros y quién los creó?

Sentencia SQL

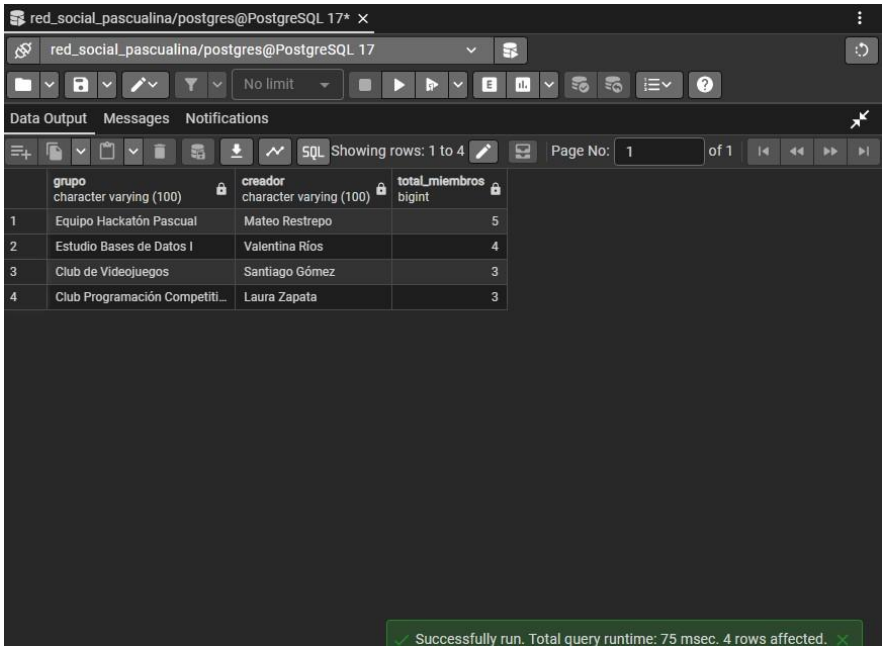
```
SELECT
    g.nombre                 AS grupo,
    u.nombre                 AS creador,
COUNT(mg.id_usuario)     AS total_miembros
FROM grupos g
JOIN usuarios u             ON u.id_usuario = g.id_usuario_creador
LEFT JOIN miembros_grupo mg ON mg.id_grupo = g.id_grupo
GROUP BY g.id_grupo, g.nombre, u.nombre
ORDER BY total_miembros DESC, grupo;
```

Explicación detallada

Une *grupos* con *usuarios* para obtener el creador y con la tabla puente *miembros_grupo* para contar los integrantes. **GROUP BY** agrupa por grupo y creador, y **ORDER BY** muestra primero los grupos más grandes. Resultado: Equipo Hackatón Pascual tiene 5 miembros, seguido de Estudio Bases de Datos I con 4.

Justificación de ALIAS

Se usa **grupo** y **creador** para distinguir los dos campos *nombre* (del grupo y del usuario) que de otro modo tendrían el mismo título en el resultado. **total_miembros** sigue la convención de conteos.



Consulta 6: Eventos con más asistentes

Pregunta de negocio

¿Qué eventos tienen más asistentes y a qué grupo pertenecen?

Sentencia SQL

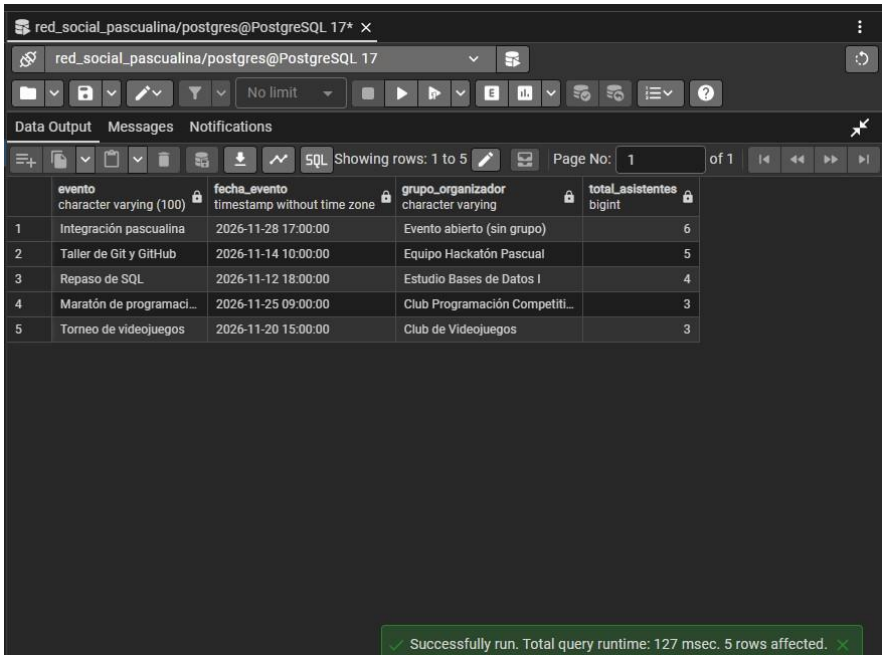
```
SELECT
    e.nombre                                          AS evento,
    e.fecha_evento                                    AS fecha_evento,
COALESCE(g.nombre, 'Evento abierto (sin grupo)') AS grupo_organizador,
COUNT(a.id_usuario)                               AS total_asistentes
FROM eventos e
LEFT JOIN grupos g       ON g.id_grupo = e.id_grupo
LEFT JOIN asistencias a  ON a.id_evento = e.id_evento
GROUP BY e.id_evento, e.nombre, e.fecha_evento, g.nombre
ORDER BY total_asistentes DESC, evento;
```

Explicación detallada

Como el campo *id_grupo* de *eventos* es opcional, se usa **LEFT JOIN** con *grupos*, y **COALESCE** reemplaza el valor nulo por el texto 'Evento abierto (sin grupo)'. Con *asistencias* se cuentan los asistentes por evento. Resultado: Integración pascualina (evento abierto) tiene la mayor convocatoria, con 6 asistentes.

Justificación de ALIAS

grupo_organizador describe el rol del grupo en el evento y **total_asistentes** sigue la convención. El texto de **COALESCE** evita mostrar celdas vacías y mejora la lectura del reporte.



Consulta 7: Estudiantes con más seguidores que el promedio

Pregunta de negocio

¿Qué estudiantes tienen más seguidores que el promedio de la red social?

Sentencia SQL

```
WITH seguidores_por_usuario AS (
SELECT
        u.nombre AS estudiante,
COUNT(s.id_seguidor) AS total_seguidores
    FROM usuarios u
    LEFT JOIN seguimientos s ON s.id_seguido = u.id_usuario
    GROUP BY u.id_usuario, u.nombre
) SELECT     estudiante,     total_seguidores,     ROUND((SELECT
AVG(total_seguidores) FROM seguidores_por_usuario), 2)
        AS promedio_seguidores
FROM seguidores_por_usuario
WHERE total_seguidores > (SELECT AVG(total_seguidores)
                          FROM seguidores_por_usuario)
ORDER BY total_seguidores DESC, estudiante;
```

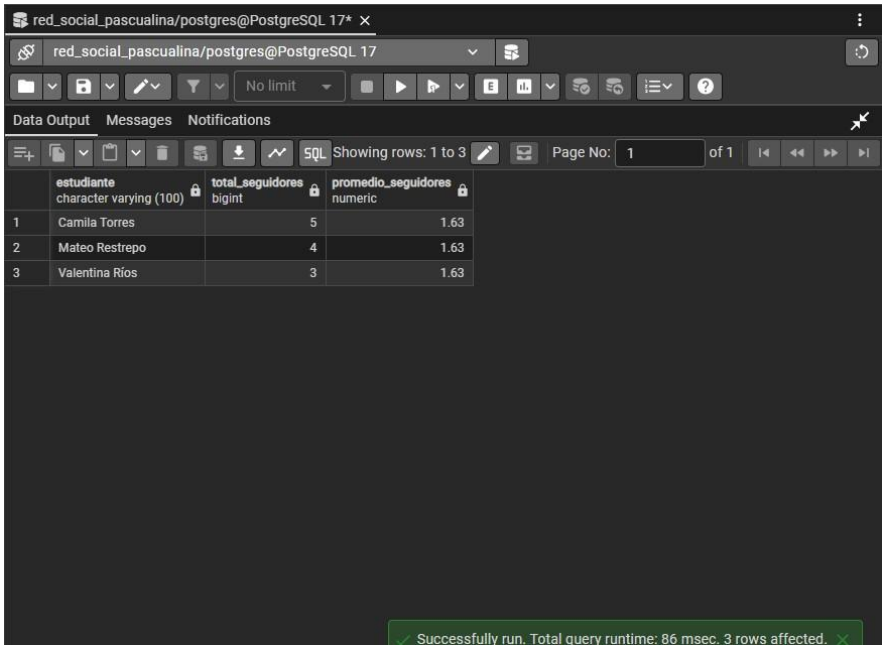
Explicación detallada

Consulta avanzada: una **CTE** calcula los seguidores de cada estudiante y se reutiliza tres veces. Una **subconsulta con AVG** obtiene el promedio de la red (13 relaciones de seguimiento entre 8 estudiantes, 1.63 aproximado) y el **WHERE** deja solo a quienes lo superan. **ROUND** limita el promedio a dos decimales. Resultado: Camila Torres, Mateo Restrepo y Valentina Ríos.

Justificación de ALIAS

promedio_seguidores usa el prefijo *promedio_* para identificar el cálculo con AVG y deja claro qué representa el valor 1.63 repetido en cada fila.

Captura del resultado



d. Vistas creadas

Las vistas no almacenan datos: guardan una consulta con nombre que se ejecuta cada vez que se consulta, por lo que siempre reflejan la información más reciente. Todas usan el prefijo **v_** y las mismas reglas de alias de la sección anterior.

Vista 1: v_perfiles_estudiantes

Objetivo de la vista

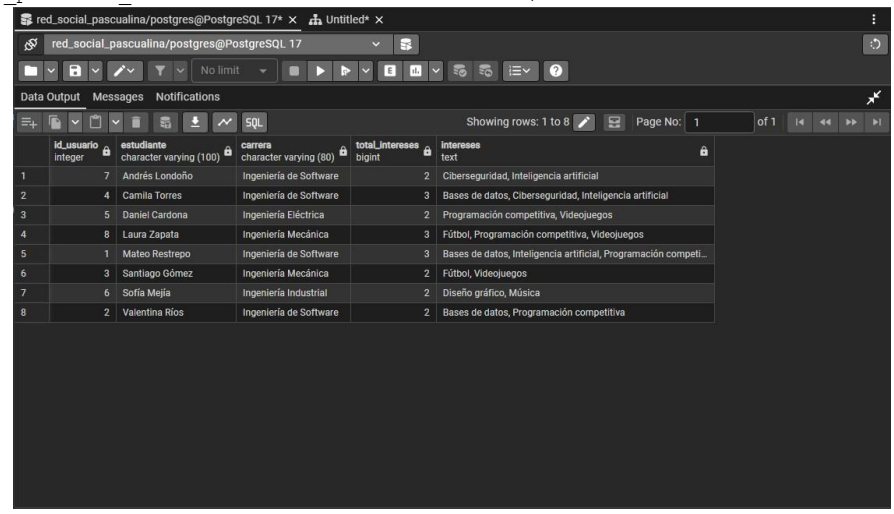
Consolida el perfil de cada estudiante (carrera) con sus intereses y habilidades en una sola fila. Es útil para buscar compañeros afines o posibles mentores sin repetir cada vez los JOIN entre tres tablas.

Sentencia SQL de creación

```
CREATE OR REPLACE VIEW v_perfiles_estudiantes AS
SELECT
    u.id_usuario                                    AS id_usuario,
    u.nombre                                        AS estudiante,
    u.carrera                                       AS carrera,
COUNT(ih.id_interes)                              AS total_intereses,
    STRING_AGG(ih.nombre, ', ' ORDER BY ih.nombre) AS intereses
FROM usuarios u
LEFT JOIN usuarios_intereses ui     ON ui.id_usuario = u.id_usuario
LEFT JOIN intereses_habilidades ih  ON ih.id_interes = ui.id_interes
GROUP BY u.id_usuario, u.nombre, u.carrera;
```

Sentencia SQL de consulta a la vista

```
SELECT * FROM v_perfiles_estudiantes ORDER BY estudiante;
```



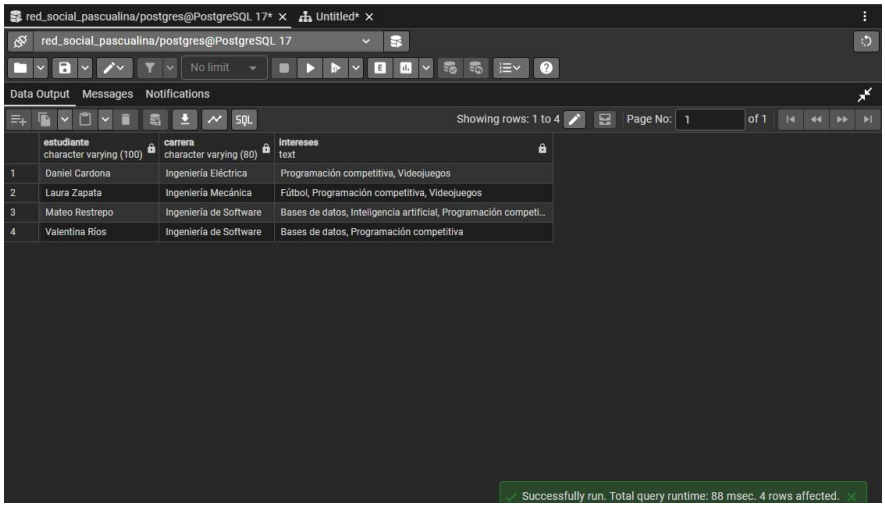
	id_usuario integer	estudiante character varying (100)	carrera character varying (80)	total_intereses bigint	intereses text
1	7	Andrés Londoño	Ingeniería de Software	2	Ciberseguridad, Inteligencia artificial
2	4	Camila Torres	Ingeniería de Software	3	Bases de datos, Ciberseguridad, Inteligencia artificial
3	5	Daniel Cardona	Ingeniería Eléctrica	2	Programación competitiva, Videojuegos
4	8	Laura Zapata	Ingeniería Mecánica	3	Fútbol, Programación competitiva, Videojuegos
5	1	Mateo Restrepo	Ingeniería de Software	3	Bases de datos, Inteligencia artificial, Programación competi…
6	3	Santiago Gómez	Ingeniería Mecánica	2	Fútbol, Videojuegos
7	6	Sofía Mejía	Ingeniería Industrial	2	Diseño gráfico, Música
8	2	Valentina Ríos	Ingeniería de Software	2	Bases de datos, Programación competitiva

Figura 10. Resultado de consultar la vista v_perfiles_estudiantes.

Consulta que reutiliza la vista

```
SELECT estudiante, carrera, intereses
```

```
FROM v_perfiles_estudiantes
WHERE intereses ILIKE '%Programación competitiva%'
ORDER BY estudiante;
```



Justificación del potencial analítico

Permite segmentar a los estudiantes por carrera e intereses, medir cuántos intereses declara cada perfil y encontrar afinidades. La función STRING_AGG junta los intereses en una sola celda, lo que facilita leer el perfil completo. Al ser una vista, otras consultas pueden filtrarla directamente; por ejemplo, para listar a los estudiantes interesados en programación competitiva (Figura siguiente), que en los datos de prueba son 4.

Vista 2: v_actividad_estudiantes

Objetivo de la vista

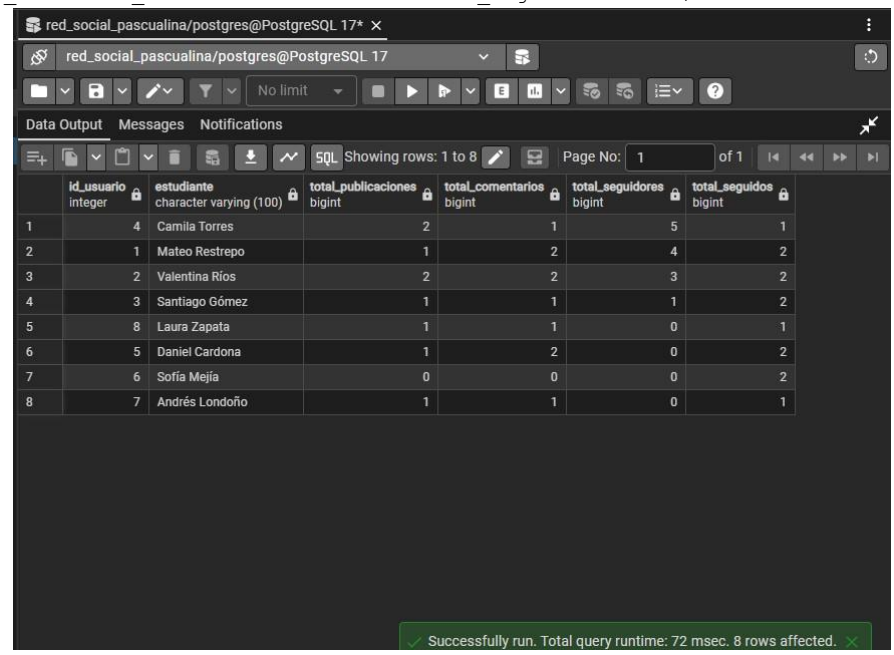
Resume la participación de cada estudiante: publicaciones, comentarios, seguidores y seguidos. Sirve para medir el compromiso con la red social.

Sentencia SQL de creación

```
CREATE OR REPLACE VIEW v_actividad_estudiantes AS
SELECT
    u.id_usuario AS id_usuario,
    u.nombre     AS estudiante,     (SELECT COUNT(*) FROM
publicaciones p       WHERE p.id_usuario_autor = u.id_usuario) AS
total_publicaciones,
    (SELECT COUNT(*) FROM comentarios c
      WHERE c.id_usuario_autor = u.id_usuario)  AS total_comentarios,
    (SELECT COUNT(*) FROM seguimientos s
      WHERE s.id_seguido = u.id_usuario)        AS total_seguidores,
    (SELECT COUNT(*) FROM seguimientos s       WHERE
s.id_seguidor = u.id_usuario)       AS total_seguidos
FROM usuarios u;
```

Sentencia SQL de consulta a la vista

```
SELECT * FROM v_actividad_estudiantes ORDER BY total_seguidores DESC;
```



	id_usuario integer	estudiante character varying (100)	total_publicaciones bigint	total_comentarios bigint	total_seguidores bigint	total_seguidos bigint
1	4	Camila Torres	2	1	5	1
2	1	Mateo Restrepo	1	2	4	2
3	2	Valentina Ríos	2	2	3	2
4	3	Santiago Gómez	1	1	1	2
5	8	Laura Zapata	1	1	0	1
6	5	Daniel Cardona	1	2	0	2
7	6	Sofía Mejía	0	0	0	2
8	7	Andrés Londoño	1	1	0	1

Justificación del potencial analítico

Es la base para analítica de participación: detectar a los estudiantes más influyentes (muchos seguidores), a los más activos y a quienes no participan, como Sofía Mejía, que en los datos de prueba no ha publicado ni comentado. Con ella la administración podría identificar mentores y diseñar acciones para activar perfiles inactivos.

Vista 3: v_resumen_grupos

Objetivo de la vista

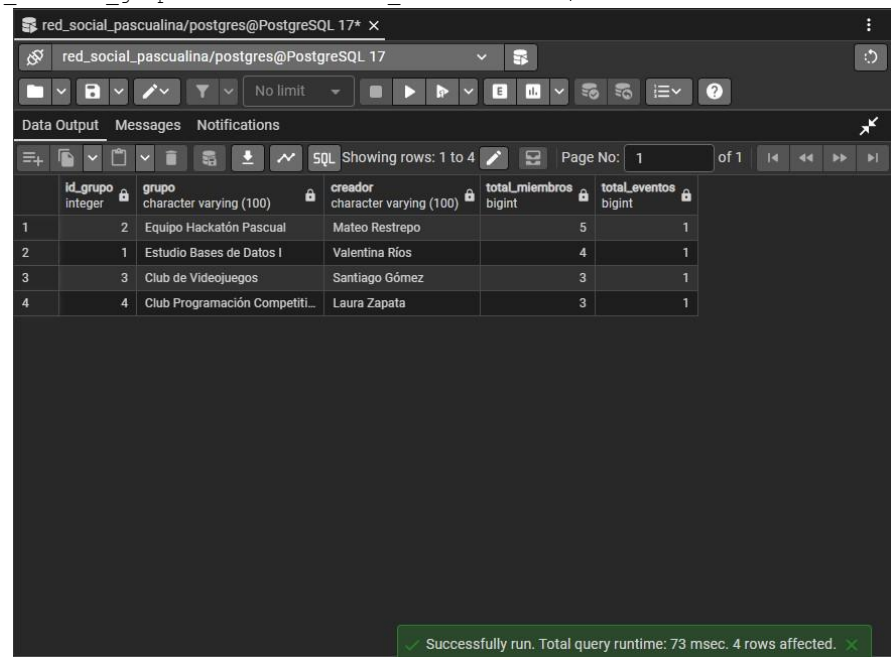
Muestra cada grupo con su creador, número de miembros y número de eventos programados. Es útil para evaluar qué grupos están más vivos.

Sentencia SQL de creación

```
CREATE OR REPLACE VIEW v_resumen_grupos AS
SELECT
    g.id_grupo   AS id_grupo,
    g.nombre     AS grupo,
    u.nombre     AS creador,     (SELECT COUNT(*) FROM
miembros_grupo mg       WHERE mg.id_grupo =
g.id_grupo) AS total_miembros,
    (SELECT COUNT(*) FROM eventos e       WHERE
e.id_grupo = g.id_grupo)  AS total_eventos
FROM grupos g
JOIN usuarios u ON u.id_usuario = g.id_usuario_creador;
```

Sentencia SQL de consulta a la vista

```
SELECT * FROM v_resumen_grupos ORDER BY total_miembros DESC;
```



Justificación del potencial analítico

Permite comparar el tamaño y la actividad de los grupos de estudio, equipos de hackatón y clubes. Un grupo con muchos miembros y pocos eventos podría necesitar apoyo para organizar actividades; uno con pocos miembros podría necesitar más difusión.

Vista 4: v_agenda_eventos

Objetivo de la vista

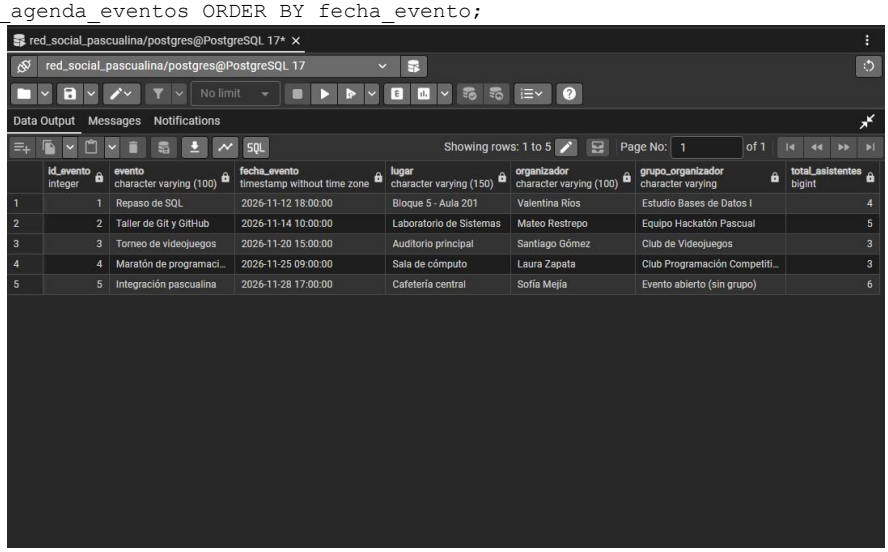
Consolida la agenda de eventos con fecha, lugar, organizador, grupo y número de asistentes, para coordinar reuniones, talleres y actividades sociales.

Sentencia SQL de creación

```
CREATE OR REPLACE VIEW v_agenda_eventos AS
SELECT
    e.id_evento                                        AS id_evento,
    e.nombre                                           AS evento,
    e.fecha_evento                                     AS fecha_evento,
    e.lugar                                            AS lugar,
    u.nombre                                           AS organizador,
COALESCE(g.nombre, 'Evento abierto (sin grupo)') AS grupo_organizador,
COUNT(a.id_usuario)                                AS total_asistentes
FROM eventos e
JOIN usuarios u         ON u.id_usuario = e.id_usuario_organizador
LEFT JOIN grupos g      ON g.id_grupo = e.id_grupo
LEFT JOIN asistencias a ON a.id_evento = e.id_evento
GROUP BY e.id_evento, e.nombre, e.fecha_evento, e.lugar, u.nombre, g.nombre;
```

Sentencia SQL de consulta a la vista

```
SELECT * FROM v_agenda_eventos ORDER BY fecha_evento;
```



	id_evento integer	evento character varying (100)	fecha_evento timestamp without time zone	lugar character varying (150)	organizador character varying (100)	grupo_organizador character varying	total_asistentes bigint
1	1	Repaso de SQL	2026-11-12 18:00:00	Bloque 5 - Aula 201	Valentina Ríos	Estudio Bases de Datos I	4
2	2	Taller de Git y GitHub	2026-11-14 10:00:00	Laboratorio de Sistemas	Mateo Restrepo	Equipo Hackatón Pascual	5
3	3	Torneo de videojuegos	2026-11-20 15:00:00	Auditorio principal	Santiago Gómez	Club de Videojuegos	3
4	4	Maratón de programaci…	2026-11-25 09:00:00	Sala de cómputo	Laura Zapata	Club Programación Competiti…	3
5	5	Integración pascualina	2026-11-28 17:00:00	Cafetería central	Sofía Mejía	Evento abierto (sin grupo)	6

Justificación del potencial analítico

Facilita medir la convocatoria de cada evento y planear espacios según la asistencia esperada. También permite comparar si los eventos de grupos convocan más o menos que los eventos abiertos (en los datos de prueba, el evento abierto tuvo la mayor asistencia: 6 personas).

e. Dificultades encontradas, ventajas y desventajas

Dificultades encontradas durante el desarrollo

Dificultad	Causa	Solución aplicada
Error al insertar datos: no se puede insertar un valor en la columna id_usuario	Las columnas de ID están definidas como identidad GENERATED ALWAYS, por lo que PostgreSQL no permite escribir el ID manualmente.	Se agregó OVERRIDING SYSTEM VALUE a los INSERT y, al final, se reajustaron las secuencias con setval para que los próximos ID automáticos no se repitan.
Error de llave duplicada en pk_usuarios	La tabla ya tenía filas (id_usuario = 1) de pruebas anteriores.	Se agregó un TRUNCATE ... RESTART IDENTITY CASCADE al inicio del script para poder ejecutarlo varias veces desde cero.
Error: no existe la relación v_perfiles_estudiantes	Se intentó consultar la vista sin haber ejecutado antes su sentencia CREATE VIEW.	Se ejecutó primero el CREATE OR REPLACE VIEW y luego el SELECT. Es importante seguir el orden creación y consulta.
Riesgo de resultados inflados al contar publicaciones y comentarios a la vez	Unir varias tablas uno a muchos en una sola consulta multiplica las filas y duplica los conteos.	Se usaron subconsultas correlacionadas y una CTE para contar cada concepto por separado (consulta 3 y vistas 2 y 3).

Ventajas y desventajas de las técnicas usadas

Técnica	Ventajas	Desventajas
JOIN con GROUP BY (consultas 1, 2, 4, 5, 6)	Código corto, fácil de leer y de optimizar. LEFT JOIN conserva las filas sin coincidencias, como intereses sin estudiantes.	Si se unen varias tablas uno a muchos, los conteos pueden duplicarse; exige cuidado al agrupar.
CTE y subconsultas (consultas 3 y 7)	Separan el cálculo en pasos con nombre, evitan el doble conteo y permiten reutilizar un resultado intermedio.	Más líneas de código; las subconsultas correlacionadas se ejecutan por cada fila y podrían ser lentas con muchos datos.
Vistas	Reutilizan consultas complejas, simplifican las consultas posteriores y centralizan la lógica de negocio en un solo lugar.	No guardan datos, por lo que recalculan cada vez; si la consulta es pesada, puede tardar. Un cambio en las tablas puede dañarlas.

f. Conclusiones y propuestas de mejora

Conclusiones individuales

- Las consultas DML sirven para responder preguntas de negocio concretas, no solo para mostrar tablas. Partir de la pregunta ayudó a decidir qué tablas unir y qué función de agregación usar.
- Entendí la diferencia entre JOIN y LEFT JOIN: con LEFT JOIN aparecen los casos sin datos, como un estudiante sin actividad o un evento sin grupo, que con un JOIN simple se perderían.

- Los errores del proceso (identidad GENERATED ALWAYS, llave duplicada y vista inexistente) me enseñaron a leer los mensajes de PostgreSQL y a seguir un orden de ejecución: datos, consultas y luego vistas.
- Las vistas ahorran trabajo: una vez creadas, se consultan con un SELECT sencillo y se pueden filtrar como si fueran una tabla.

Conclusiones generales del proceso

El modelo físico de la red social soporta consultas útiles para la analítica: afinidad entre estudiantes, identificación de mentores, participación, tamaño de grupos y convocatoria de eventos. Como la tarea se realizó de forma individual, no hubo una discusión de grupo que contrastar; las conclusiones corresponden al trabajo y aprendizaje del estudiante.

Propuestas de mejora

- Participar en el foro desde el inicio de la unidad y no al final, planificando el tiempo con los demás compromisos, para recibir retroalimentación antes de entregar.
- Crear índices sobre las claves foráneas más consultadas (por ejemplo, id_usuario_autor y id_publicacion) y evaluar las vistas materializadas si el volumen de datos crece.
- Agregar consultas que usen el campo JSON de eventos (detalles_json) y una fecha en los comentarios, para analizar la actividad a lo largo del tiempo.
- Probar las consultas con un volumen de datos mayor para medir su rendimiento y aplicar técnicas de optimización.

Organización del repositorio Git

```
Tarea5/
|-- scripts/
|   |-- 01_datos_prueba.sql
|   `-- 02_consultas_y_vistas.sql
|-- informe/
|   `-- Informe_Tarea5.pdf
`-- video/
    `-- enlace_youtube.txt
```